Problem 1

a. (0, 1), (-2, -1), (0.5, -0.5)

b. We use $d^t = \frac{|w^T x^t + w_0|}{||w||}$ where $w^T = [1, -1]$ We pick samples (0, 1), (-2, -1), and (1, 3) and calculate their distance from $x_1 = x_2 = 0$

$$||w|| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$$

Area of the triangle divided by the base on the line.

For (0, 1), the distance is $d_t = \frac{w^T x^t}{||w||} = \frac{|0*1+1*-1|}{\sqrt{2}} = \frac{1}{\sqrt{2}}$. In fact, we find that for any point on the support vector, the distance to the hyperplane is $\frac{1}{\sqrt{2}}$ and this holds true for (-2, -1).

For (1, 3), the distance is $d_t = \frac{|1*1+3*-1|}{\sqrt{2}} = \frac{2}{\sqrt{2}}$

c. Yes for (0, 1) and no for (-1, -2.5)

d. Yes the boundary changes as the support vectors will change.

e. The SVM method considers two main objective functions: one maximizes the margin while the other, with coefficient C, accounts for error. A higher C puts more weight on the significance of error within the SVM while a lower C makes error less significant.

f. In real world application, a linear hard margin SVM method would be used on only the most simple of classification problems; namely those with data that doesn't have much noise or complicated non-linear representations. If the data is susceptible to noise, a soft margin SVM method would work better. However, there are situations where even noise cannot account for the inflexible fit that a linear SVM offers. In this case, we use a kernel SVM to project to a higher dimension that can better represent the data.

Problem 2

a.

When choosing the root attribute we can consider temperature, humidity, or outlook.

Starting with outlook, there are three branches:

Rainy: ○○○○● Overcast: ○○● Sunny: ●●●●●●●

And we calculate impurity as

$I'_{outlook} = -[\frac{1}{3}(\frac{1}{5}log_2(\frac{1}{5}) + \frac{4}{5}log_2(\frac{4}{5})) + \frac{1}{5}(\frac{1}{3}log_2(\frac{1}{3}) + \frac{2}{3}log_2(\frac{2}{3})) + \frac{7}{15}(1 * log_2(1) + 0 * log_2(0))]$
$I'_{outlook} = 0.4243$

Starting with temperature, there are three branches:

Cool: ○○○●●● Mild: ○○●●●● Hot: ○●●

And we calculate impurity as

$I'_{temperature} = -[\frac{2}{5}(\frac{1}{2}log_2(\frac{1}{2}) + \frac{1}{2}log_2(\frac{1}{2})) + \frac{2}{5}(\frac{2}{3}log_2(\frac{2}{3}) + \frac{1}{3}log_2(\frac{1}{3})) + \frac{1}{5}(\frac{2}{3}log_2(\frac{2}{3}) + \frac{1}{3}log_2(\frac{1}{3}))]$
$I'_{temperature} = 0.95098$

Starting with humidity, there are two branches:

Normal: ○○○●●●●●● High: ○○○○●●

And we calculate impurity as

$I'_{humidity} = -[\frac{3}{5}(\frac{7}{9}log_2(\frac{7}{9}) + \frac{2}{9}log_2(\frac{2}{9})) + \frac{2}{5}(\frac{1}{3}log_2(\frac{1}{3}) + \frac{2}{3}log_2(\frac{2}{3}))]$ $I'_{humidity} = 0.8258$

We see that outlook has the lowest impurity measure, so we begin building the decision tree with outlook as the root. An outlook of 'sunny' is automatically a 'yes', so we focus on splitting the 'overcast' branch first. Splitting by temperature gives us...

Cool: ○ Mild: ○ Hot: ●

Which has an impurity measure of $I'_{temperature} = 0$. Which is the lowest possible, so we don't need to consider the alternative of splitting by 'humidity'.

Splitting the 'rainy' branch has two options. Splitting by temperature first gives us...

Cool: ○○ Mild: ○● Hot: ○

Then $I'_{temperature} = -\frac{2}{5}(\frac{1}{2}log_2(\frac{1}{2}) + \frac{1}{2}log_2(\frac{1}{2}))$

Splitting by humidity first gives us...

Normal: ○● High: ○○○

Then $I'_{humidity} = -\frac{2}{5}(\frac{1}{2}log_2(\frac{1}{2}) + \frac{1}{2}log_2(\frac{1}{2}))$

Since both of the splits yield the same impurity measure it is equally valid to split the 'rainy' branch of 'outlook' by either humidity or temperature first, and then split by the other one.

I will split by temperature first and then humidity.

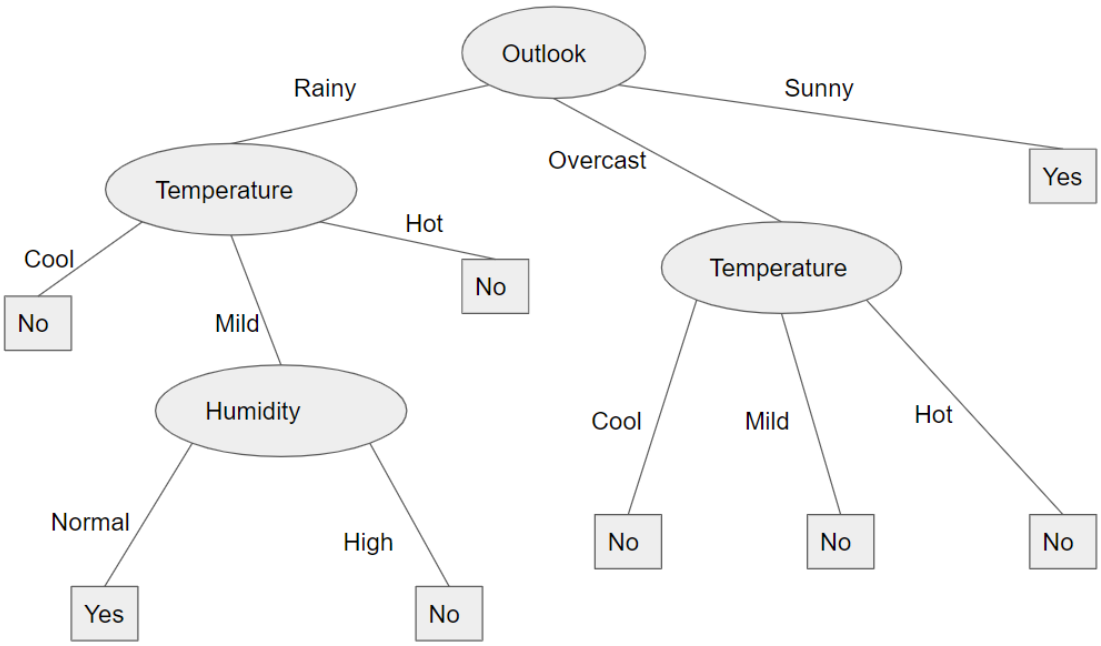


Figure 1: Decision tree

b. overcast and cool is a No

Problem 3

a. **Report the training and validation accuracy by different θ. What θ should you use? Report the accuracy on the test set using this θ.**

```
(.penv) luke@DESKTOP-MQ678JB:/mnt/c/Users/lukec/Documents/Coursework/CSCI5521/homework/hw4$ python3 hw4.py
Training/validation accuracy for minimum node entropy 0.010000 is 1.000 / 0.863
Training/validation accuracy for minimum node entropy 0.050000 is 0.999 / 0.863
Training/validation accuracy for minimum node entropy 0.100000 is 0.997 / 0.865
Training/validation accuracy for minimum node entropy 0.200000 is 0.990 / 0.867
Training/validation accuracy for minimum node entropy 0.500000 is 0.963 / 0.863
Training/validation accuracy for minimum node entropy 0.800000 is 0.919 / 0.856
Training/validation accuracy for minimum node entropy 1.000000 is 0.871 / 0.838
Training/validation accuracy for minimum node entropy 2.000000 is 0.596 / 0.600
Test accuracy with minimum node entropy 0.200000 is 0.872
```

Figure 2: results

From this figure we can see that $\theta = 0.200000$ has the highest validation accuracy of 0.872

b. **What can you say about the model complexity of the Decision Tree, given the training and validation accuracy? Briefly explain.**

As our minimum entropy threshold increases our accuracy increases as well up until $\theta = 0.200000$. After this level, the further raising of the entropy threshold causes accuracy to go down. The lower the entropy threshold, the more complex the decision tree model is because for attribute splitting is needed to reach the lower entropy.